

Activity 05 Critical Thinking

Emmanuel Gohourou

CSC 4360 Monday and Wednesday

Question 1 History is a claim about time

My history should represent counter values that actually changed. The app starts with `_counter` at 40, `_increment` at 7, an empty `_history`, and controller text of 7. In my four-action trace, Increase changes 40 to 47 and saves 40. Moving the slider directly to 149 saves 47. Another Increase would produce 156, so `_moveTo` rejects it. Undo then restores 47 and removes that last saved value. The increment and controller text stay at 7 throughout this sequence.

The transition table records every application state field before and after each action. The slider and counter color aren't separate stored values. `build` reads the counter to draw the slider, and `_counterColor` chooses its color. This keeps those parts of the screen consistent when Undo changes the counter.

I save history for valid increases, decreases, resets that change the value, and changed whole-number slider positions. Each slider position counts as a change, so a long drag can take several undos to retrace. I wouldn't add entries for invalid text, rejected moves, or the same value again. Changing the increment doesn't belong in this counter-only history either. Undo restores the counter, not the increment field.

One plausible mistake is saving history whenever the slider calls its method, even when the rounded value hasn't changed. The starter allows this. After reaching 47, repeating 47 adds another 47 to history. Undo then leaves the display at 47, which makes it look like nothing happened. The repair is the same-value check in `_moveTo` before `_history.add`. `_undo` still uses `removeLast`, so it restores the most recent real change. An empty history only shows a message and doesn't change anything. The starter failure was reproduced in a separate Flutter test, and the Android tests confirmed that the repaired app ignores repeated values.

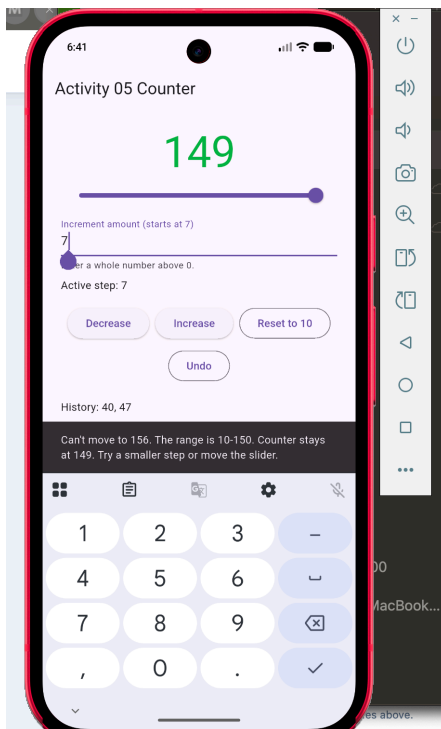
Question 1 Transition table

State = (counter, increment, history, controller text). The same four fields appear before and after every action.

Action	Before	After
Increase	(40, 7, [], "7")	(47, 7, [40], "7")
Slider directly to 149	(47, 7, [40], "7")	(149, 7, [40, 47], "7")
Increase rejected	(149, 7, [40, 47], "7")	(149, 7, [40, 47], "7")
Undo	(149, 7, [40, 47], "7")	(47, 7, [40], "7")

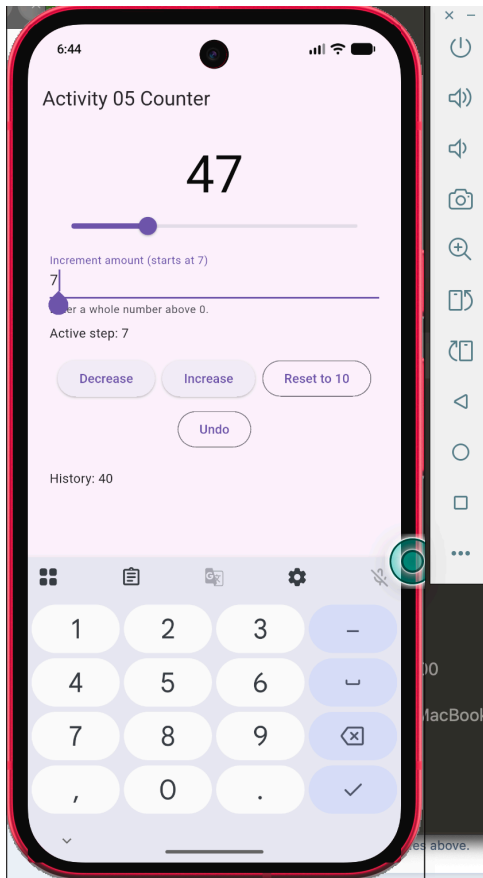
Code references: `_moveTo`, `_readIncrement`, `_undo`, `_counterColor`, and the Slider callback in `app/lib/main.dart`. The failed history policy is reproduced by `app/test/history_policy_test.dart` using the unchanged starter.

Capture 1: rejected Increase leaves 149 and its history unchanged.



Visible warning: Can't move to 156. The range is 10-150. Counter stays at 149. Try a smaller step or move the slider.

Capture 2: Undo restores 47 and removes the last history entry.



The displayed counter, slider position, and history must agree. `_increment` stays 7 and the controller text stays "7" through this trace.

Question 2 Constraint feedback changes behavior

A boundary error and an input error need different explanations. At 149 with an active step of 7, Increase tries to reach 156. The number is valid, but the move breaks the counter's maximum of 150. My boundary message names 156, explains the 10-150 range, and says the counter stays at 149. It also suggests a smaller step or the slider. The user can still decrease the counter or undo an earlier change.

An invalid increment is different because it doesn't describe an allowed step. `_readIncrement` uses `int.tryParse` and checks that the result is above zero. Blank input, a decimal, a negative number, and a word are rejected. The previous valid increment stays active, while the typed text remains in the field so it can be corrected. Neither the counter nor its undo history changes just because the field contains invalid text.

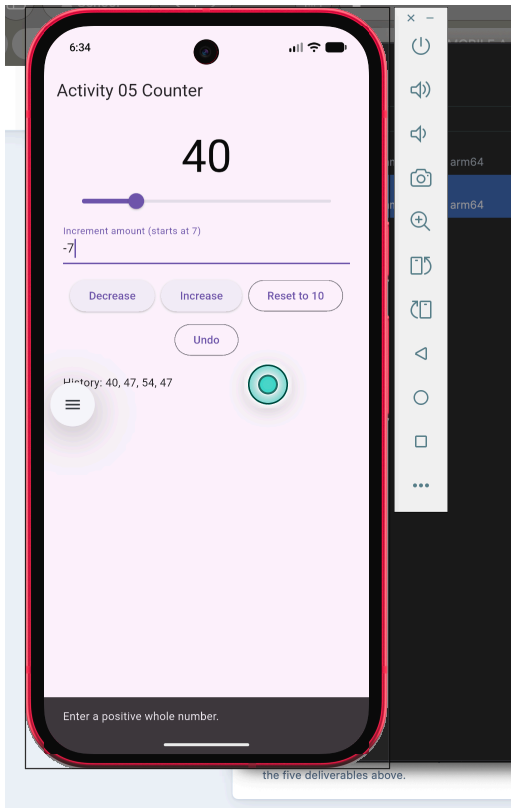
My peer tried 2.6, -7, and toast. They saw the warning asking for the correct input. That is the observation I have; I can't say they understood everything just because the warning appeared. The original message was Enter a positive whole number. After that observation, I revised it to give an example, reject decimals and words clearly, and state which step is still active. The before-and-after captures show the message change.

I also added the Active step line below the field. This is my UI improvement because invalid text can remain visible while a different valid number controls the buttons. The line helps explain that situation after the `SnackBar` disappears. `_showMessage` replaces the current `SnackBar` so older warnings don't form a long queue. These changes make the next action clearer without changing the counter rules or adding rejected actions to history.

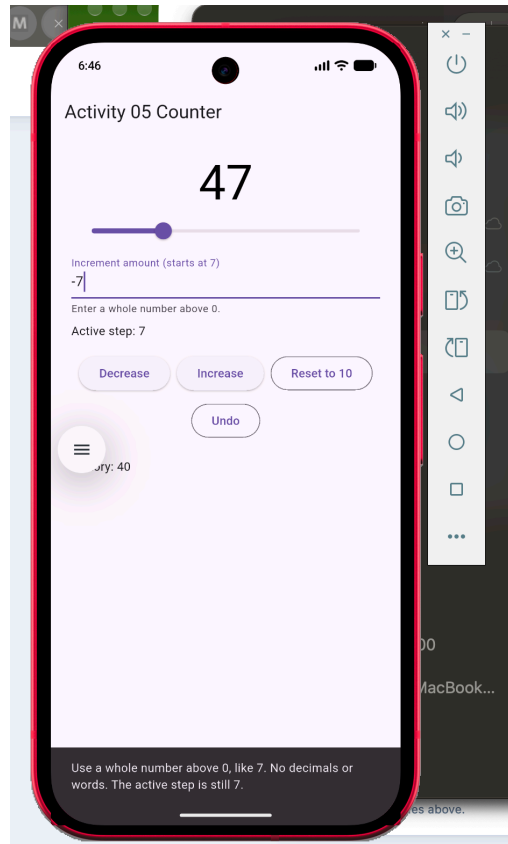
Code references: `_readIncrement`, `_showMessage`, `_moveTo`, and the Active step text in `app/lib/main.dart`. Peer observation: inputs 2.6, -7, toast; warning asking for correct input appeared.

Question 2 Before and after message evidence

Before: starter input warning.



After: revised input warning shows an example and retained step.



Before message: Enter a positive whole number.

After message: Use a whole number above 0, like 7. No decimals or words. The active step is still 7.

The boundary violation appears in Capture 1. It identifies the attempted value and the allowed range. The input warning instead explains acceptable input and which step remains active.